

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1

Case Study

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Assessment Tool Weightage: 25%

QUESTIONS:

Total Marks: 25

Case Study 1: CNN for Automated Medical Image Classification

A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.

Answer:

A suitable CNN architecture for classifying chest X-ray images into **normal** and **disease-affected** categories should balance feature-extraction capability with computational efficiency, since medical images are large and detail-sensitive. The proposed architecture is as follows:

- **Input Layer:** X-ray images resized to a fixed dimension (e.g., 224 x 224 x 1 for grayscale, or 224 x 224 x 3 if converted to RGB), with pixel values normalized to [0,1].

- **Convolutional Block 1:** 32 filters of size 3x3 with ReLU activation, followed by a 2x2 max-pooling layer to detect low-level features such as edges and boundaries of lung regions.
- **Convolutional Block 2:** 64 filters of size 3x3 with ReLU activation and 2x2 max-pooling, to capture mid-level features such as textures and localized opacities.
- **Convolutional Block 3:** 128 filters of size 3x3 with ReLU activation and 2x2 max-pooling, to learn high-level, disease-specific patterns such as nodules or infiltrates.
- **Batch Normalization** after each convolutional block to stabilize and speed up training.
- **Flatten / Global Average Pooling Layer** to convert the final feature maps into a one-dimensional feature vector while limiting the number of parameters.
- **Fully Connected Layer:** a dense layer with 128 neurons and ReLU activation to combine extracted features for classification.
- **Dropout Layer** (rate 0.5) placed before the output layer to reduce overfitting.
- **Output Layer:** a single neuron with a sigmoid activation function (for binary classification of normal vs disease-affected), or two neurons with softmax if treated as a two-class categorical problem.

This layered design progressively transforms raw pixel intensities into abstract, disease-relevant representations, allowing the network to automatically learn discriminative features without manual feature engineering.

2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.

Answer:

Motivation for Using CNNs

Traditional machine learning approaches for X-ray classification require manually engineered features (e.g., edge detectors, texture descriptors), which are time-consuming to design and often fail to generalize across variations in image size, brightness, and quality. CNNs overcome this limitation because they can automatically learn hierarchical visual features directly from raw pixel data, are translation-invariant (a diseased region is detected regardless of its position in the image), and use far fewer parameters than a fully connected network of comparable depth, which makes them well suited to large medical imaging datasets.

Role of Each Layer

- **Convolutional Layers:** Apply learnable filters (kernels) that slide across the image to extract local patterns such as edges, textures, and abnormal tissue regions. Early layers capture generic features, while deeper layers capture disease-specific structures.
- **Filters:** Each filter acts as a specialized feature detector; a set of filters in one layer produces multiple feature maps, allowing the network to simultaneously detect various patterns (e.g., boundaries, opacities).
- **Pooling Layers:** Down-sample the feature maps (typically using max-pooling), reducing spatial dimensions and computational cost while retaining the most salient features and providing a degree of translation invariance.

- **Fully Connected Layers:** Positioned toward the end of the network, they combine the high-level features extracted by the convolutional and pooling layers to make the final classification decision (normal vs disease-affected).
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.

Answer:

Parameter sharing is the principle by which the same filter (a small set of weights) is used across every spatial location of the input image, rather than learning a separate weight for every pixel connection as a fully connected (FC) network would.

Illustrative comparison: Consider a 224 x 224 grayscale X-ray image feeding into the first hidden layer.

- In a fully connected layer with, say, 128 neurons, the number of trainable weights would be $224 \times 224 \times 128 = 6,422,528$ weights, all learned independently.
- In a convolutional layer using 32 filters of size 3x3, the number of trainable weights is only $3 \times 3 \times 32 = 288$ weights (plus 32 bias terms), because the same 3x3 filter is reused ("shared") at every location as it slides across the entire image.

This dramatic reduction occurs because each filter learns one generalizable pattern (e.g., an edge or a texture) that is assumed to be useful anywhere in the image, so it does not need a unique weight for every pixel position. The benefits are threefold: (1) it drastically reduces the number of trainable parameters, lowering memory and computation requirements; (2) it reduces the risk of overfitting, which is critical given limited labeled medical data; and (3) it gives the network translation invariance, so a disease pattern is detected consistently regardless of where it appears in the X-ray.

4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.

Answer:

Because medical imaging datasets are often limited in size and diversity, the proposed CNN is prone to overfitting, where it memorizes training examples instead of learning generalizable disease patterns. The following regularization techniques are recommended:

- **Dropout:** Randomly deactivates a fraction of neurons (e.g., 30-50%) during each training iteration, forcing the network to learn redundant, robust representations instead of relying on specific neurons, which reduces co-adaptation and overfitting.
- **Data Augmentation:** Artificially expands the training set by applying label-preserving transformations such as rotation, horizontal flipping, zooming, brightness/contrast adjustment, and slight translation to the X-ray images. This exposes the model to greater variability, directly addressing the dataset's variations in size, brightness, and image quality, and improves generalization to unseen scans.
- **Batch Normalization:** Normalizes the activations of each layer within a mini-batch, which stabilizes and accelerates training, allows the use of higher learning rates, and has a mild regularizing effect that reduces the network's sensitivity to specific weight initializations.

Used together, these techniques allow the network to achieve strong performance on unseen X-ray images rather than simply memorizing the training set, which is essential for a clinically reliable diagnostic system.

5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

Answer:

Recommendation: ResNet is more appropriate than AlexNet for this medical image classification application. The justification is as follows:

- **Feature Learning:** ResNet's residual (skip) connections allow the network to learn very fine-grained, hierarchical features across many layers without the signal degrading, which is important for detecting subtle abnormalities (e.g., small nodules or early-stage infiltrates) in chest X-rays that AlexNet's shallower architecture may miss.
- **Network Depth:** AlexNet has only 8 weighted layers, limiting the complexity of features it can represent. ResNet architectures (e.g., ResNet-18/34/50) can go far deeper (18 to over 100 layers) because residual connections mitigate the vanishing-gradient problem that would otherwise prevent very deep networks from training effectively, enabling richer, more discriminative disease-specific representations.
- **Computational Requirements:** ResNet is more computationally intensive than AlexNet due to its greater depth, requiring more memory and training time. However, for a healthcare application where diagnostic accuracy is critical and inference is not required in strict real time (unlike autonomous driving), this additional computational cost is an acceptable trade-off, and transfer learning with pretrained ResNet weights can reduce training time considerably.
- **Classification Performance:** Empirically, ResNet consistently outperforms AlexNet on complex image classification benchmarks due to its depth and better gradient flow, translating to higher sensitivity and specificity in distinguishing normal from disease-affected X-rays, which is essential to minimize missed diagnoses (false negatives) in a clinical setting.

In summary, although AlexNet is simpler and faster to train, ResNet's superior feature-learning capacity and robustness to vanishing gradients make it the more suitable choice where diagnostic accuracy takes precedence over inference speed.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.

Answer:

The autonomous vehicle vision system must reliably recognize multiple object categories (cars, pedestrians, traffic signs, bicycles, road obstacles) under varying lighting, viewing angles, distances, and road environments, and must do so with low latency for real-time driving decisions. A suitable CNN-based architecture is designed as follows:

- **Input Layer:** Camera images resized to a fixed resolution (e.g., 224 x 224 x 3 RGB), normalized for consistent brightness and contrast to reduce the impact of varying lighting conditions.
- **Convolutional Feature-Extraction Backbone:** A stack of convolutional blocks (convolution + ReLU + pooling), increasing in filter depth (e.g., 32 to 64 to 128 to 256), that progressively extract low-level features (edges, colors) in early layers and high-level, object-specific features (vehicle shapes, pedestrian silhouettes, sign geometry) in deeper layers.
- **Batch Normalization** after each convolutional block to maintain stable training across the diverse conditions found in road imagery.
- **Multi-Scale Feature Processing:** Because objects appear at different distances and scales (a nearby pedestrian vs. a distant traffic sign), pooling and multiple convolutional stages allow the network to capture features at multiple spatial resolutions.
- **Fully Connected / Output Layers:** Dense layers that combine the extracted features, followed by a softmax output layer sized to the number of object classes (cars, pedestrians, traffic signs, bicycles, obstacles, background), enabling multi-class recognition.

This architecture is designed to be trained on a large, diverse dataset of road scenes so that the shared convolutional filters generalize across the wide range of lighting, angle, and distance variations described in the case study.

2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.

Answer:

- **Input Layer:** Accepts the raw camera image and passes pixel intensity values into the network; consistent preprocessing (resizing, normalization) at this stage ensures the network receives standardized data despite varying camera conditions.
- **Convolution Layer:** Applies learnable filters across the image to detect spatially local patterns, producing feature maps that highlight the presence of specific visual structures at each location.
- **Activation Layer (ReLU):** Introduces non-linearity by zeroing out negative activations, enabling the network to model complex, non-linear relationships between pixels and object categories rather than being limited to linear combinations.
- **Pooling Layer:** Down-samples feature maps (typically via max-pooling), reducing spatial resolution and computation while retaining the strongest activations, and adding robustness to small shifts in object position, which is useful as vehicles and pedestrians move within the frame.

- **Fully Connected / Output Layer:** Integrates the high-level features produced by the convolutional stack and maps them to class probabilities for each object category (car, pedestrian, traffic sign, bicycle, obstacle).

Progressive Feature Learning

As the input passes through successive convolutional layers, the filters learn an increasingly abstract hierarchy of features. The earliest layers detect **low-level features** such as edges, corners, and color gradients, common to all object types. Intermediate layers combine these into **mid-level features** such as contours, wheel shapes, or limb-like structures. The deepest layers combine these into **high-level features** that represent whole objects or object parts, such as a car's silhouette, a pedestrian's posture, or a traffic sign's shape and symbol, which the fully connected/output layers then use to make the final classification decision.

3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.

Answer:

Parameter sharing means that each convolutional filter uses one fixed set of weights that is applied ("convolved") across every spatial location of the input image, instead of learning a unique weight for every pixel position as a fully connected layer would.

Importance in This Application

- **Reducing Computational Complexity:** Since road scenes are captured at high resolution and must be processed continuously in real time, parameter sharing keeps the number of trainable weights small (proportional to filter size, not image size), which reduces memory usage and speeds up both training and inference, an essential requirement for on-board vehicle computing hardware.
 - **Improving Translation-Related Feature Detection:** A pedestrian, vehicle, or traffic sign can appear anywhere in the camera frame and at varying distances. Because the same filter is applied across the entire image, the network can detect a given object (e.g., a car) with equal sensitivity whether it appears on the left, right, center, near, or far part of the frame, without needing to separately learn that feature for every possible position. This translation invariance is critical for reliable, position-independent object recognition on the road.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.

Answer:

The CNN model is at risk of overfitting because road-scene data can vary enormously (weather, lighting, viewing angle, occlusion), and a network with many parameters may memorize specific training examples rather than learning generalizable object representations, resulting in poor performance on new, unseen driving conditions. Suitable regularization methods include:

- **Data Augmentation:** Applying transformations such as brightness/contrast variation, random cropping, horizontal flipping, rotation, and simulated weather or occlusion effects to artificially expand the training data and expose the model to the same lighting, angle, and distance variability described in the case study.
- **Dropout:** Randomly deactivating a subset of neurons in the fully connected layers during training to prevent the network from becoming overly reliant on

specific feature combinations, encouraging more robust, generalizable representations.

- **Batch Normalization:** Normalizing layer activations during training to stabilize learning, allow higher learning rates, and provide a mild regularizing effect that reduces sensitivity to variations between batches of road images.
 - **Early Stopping and Weight Decay (L2 Regularization):** Monitoring validation performance to halt training before the model starts to overfit, and penalizing overly large weights to keep the model simpler and more generalizable.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.

Answer:

- **Architecture:** AlexNet uses a relatively simple, sequential stack of 5 convolutional layers followed by 3 fully connected layers. ResNet introduces residual (skip) connections that allow the input of a block to be added directly to its output, enabling the training of much deeper networks.
 - **Depth:** AlexNet has 8 weighted layers in total. ResNet variants range from 18 layers (ResNet-18) to over 150 layers (ResNet-152), allowing ResNet to capture significantly more complex and hierarchical visual patterns needed to distinguish cars, pedestrians, cyclists, and signs under diverse conditions.
 - **Parameter Efficiency:** AlexNet has around 60 million parameters, a large share of which are concentrated in its fully connected layers. ResNet, despite being deeper, uses global average pooling instead of large fully connected layers and is generally more parameter-efficient per unit of representational power (e.g., ResNet-50 has around 25 million parameters while achieving higher accuracy than AlexNet).
 - **Training Difficulty:** AlexNet is easier and faster to train because of its shallower structure, but its representational capacity is limited. ResNet would be very difficult to train as a plain deep network because gradients would vanish across many layers; its residual connections specifically solve this problem, making deep training feasible and stable.
 - **Vanishing-Gradient Problem:** AlexNet, being shallow, is less affected by vanishing gradients but cannot go much deeper without encountering this issue. ResNet's skip connections allow gradients to flow directly through identity shortcuts during backpropagation, effectively solving the vanishing-gradient problem and enabling stable training of very deep networks.
 - **Feature-Learning Capability:** Due to its greater depth and stable gradient flow, ResNet learns richer, more discriminative hierarchical features than AlexNet, which translates into stronger performance on complex, multi-class object recognition tasks such as identifying cars, pedestrians, traffic signs, bicycles, and obstacles under varied real-world conditions.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

Answer:

Recommendation: ResNet (e.g., a ResNet-18 or ResNet-34 variant) is the most suitable architecture for the autonomous vehicle object recognition system, with the following justification:

- **Accuracy:** Autonomous driving decisions directly affect passenger and pedestrian safety, so classification accuracy is the top priority. ResNet's greater depth and residual connections allow it to learn more discriminative features, giving higher accuracy in distinguishing visually similar categories (e.g., pedestrians vs. cyclists, or different traffic sign types) compared with the shallower AlexNet.
- **Computational Cost:** While ResNet is deeper than AlexNet, its use of global average pooling and efficient residual blocks keeps its parameter count reasonable, and smaller variants such as ResNet-18 offer a strong accuracy-to-computation trade-off. This cost can be further reduced using techniques such as model quantization, pruning, or deployment on dedicated GPU/edge-AI hardware commonly used in autonomous vehicles.
- **Real-Time Application Requirements:** Although deeper networks are typically slower than shallow ones, ResNet's efficient design (compared to AlexNet's large fully connected layers) and compatibility with hardware acceleration make it feasible to achieve real-time inference speeds. The improvement in safety-critical accuracy justifies the moderate additional computational investment, whereas AlexNet's lower accuracy poses a greater risk in complex, safety-critical driving scenarios.

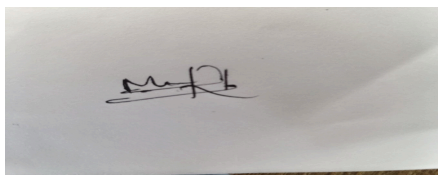
Overall, ResNet offers the best balance of high recognition accuracy, manageable computational cost through efficient architecture design, and the depth needed for robust, real-time performance in the safety-critical context of autonomous driving.

Code of Conduct:

I Maniyar Mohammad Rehan (192425391) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a white piece of paper with a handwritten signature in black ink. The signature is cursive and appears to read 'Maniyar'.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand